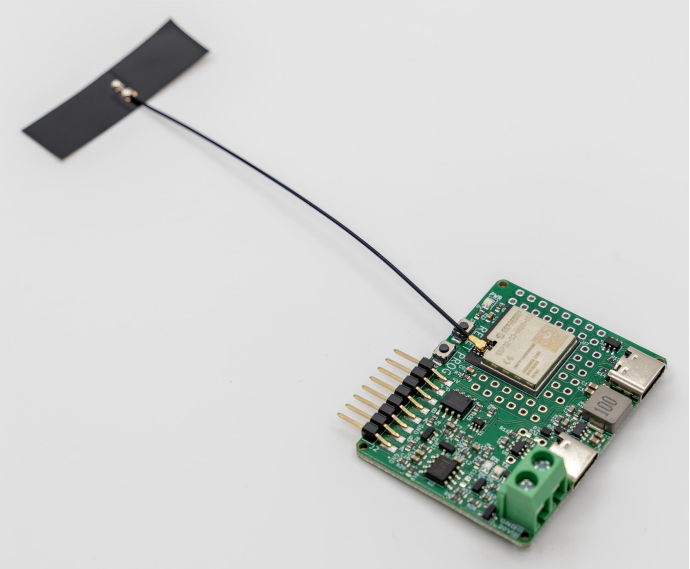
PROGRAMMABLE MOTION CONTROL

RS485 Controller

ESP32-S3 development board for Gearotons servomotors and other RS485 devices

Affordable and Simple All-in-One Motion Control

From Education to Innovation





Actual product photographs. External antenna included; cables, power supply and motors are separate system components.

11.7 g
Including attached antenna

230400 baud
RS485 example setting

PCB 0.5
Hardware covered here

A compact controller for shared-bus motion

Run motor-control logic directly on the ESP32-S3. The board combines native USB programming, a half-duplex RS485 interface, a USB-C PD power input and accessible GPIO pads in a compact open PCB.

Embedded control: dual-core LX7 processor, up to 240 MHz; 2.4 GHz Wi-Fi and Bluetooth 5 LE capabilities depend on application firmware.

Simple wiring: two parallel A/B/GND groups, automatic RS485 direction circuitry, reset and programming buttons.

Motor-power integration: 20 V USB-C PD request in the standard assembly, with a screw-terminal power output.

For robotics, automation prototypes, teaching and distributed motion systems. This draft describes hardware capabilities; installed firmware determines application behavior.

SPECIFICATIONS

Electrical & power

Design values are distinguished from measurements and qualified product limits.

Parameter	Specification / conditions
Processor / module	ESP32-S3-MINI-1U; dual-core Xtensa LX7, up to 240 MHz.
Wireless hardware	2.4 GHz Wi-Fi 802.11b/g/n and Bluetooth 5 LE; external antenna included. Wireless control requires suitable firmware.
Programming supply, J5	5 V nominal USB input. Native ESP32-S3 USB data connection; motor power is supplied separately.
PD power input, J3	Standard design requests 20 V using CH221K and R27 = 200 kΩ. Actual voltage depends on the supply and successful PD negotiation.
Motor-power output, J1	Pin 1 = J3 VBUS rail (labelled +20 V); pin 2 = GND. This is a direct pass-through, not a separately regulated 20 V output.
Logic / RS485 interface	3.3 V logic; SP3485EN half-duplex differential transceiver. Controller, USB and motor grounds are common; no galvanic isolation.
RS485 example setting	230400 baud, 8 data bits, no parity, 1 stop bit. TX = GPIO4; RX = GPIO5. Board maximum baud rate is not specified.
Bus loading / protection	R5 = 220 Ω across A/B. R1 = 560 Ω pulls A high; R3 = 560 Ω pulls B low. SM712 transient suppressor on the bus.
Auxiliary power pins, J2	3.3 V rail and J5-associated 5 V rail. Available external current is not specified. The 5 V rail is not generated from the PD input.

Power routing

J5 · PROGRAM

5 V USB + native USB data

3.3 V logic

Diode-fed buck supply

J3 → J1

PD rail → motor supply

Verify motor power before connecting

Use a USB-C PD supply that explicitly offers 20 V and a suitable cable. Measure J1 before attaching motors; a non-PD or unsuccessful source may leave the rail at a lower voltage. J5 alone does not provide motor power at J1.

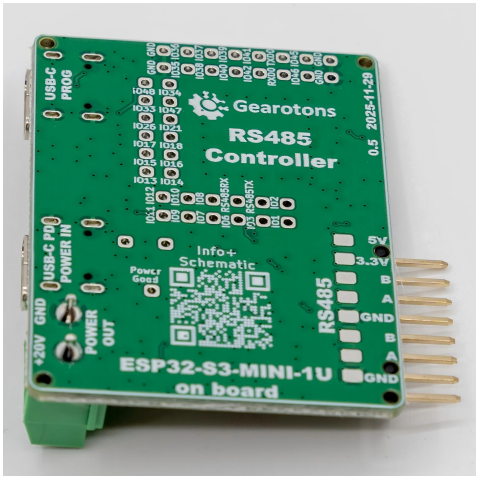
Not yet specified: board current consumption, continuous motor-power pass-through current, maximum cable length, operating/storage temperature and humidity limits. Component ratings are not board ratings. Product-level ESD/EMC ratings have not been established.

INTERFACES

Connectors & controls

Match the printed PCB labels before attaching wires. The two bus groups are electrically parallel.

J2 pin	Signal	Use
1	GND	Common ground
2	A	RS485 A
3	B	RS485 B
4	GND	Common ground; same net as pin 1
5	A	Same bus as pin 2
6	B	Same bus as pin 3
7	3.3 V	Logic supply rail; external load not rated
8	5 V	Programming USB supply rail; external load not rated



Label-side orientation
USB-C sockets to the left, J2 to the right. Reading J2 from the bottom upward: pins 1 through 8.

Header pitch: 2.54 mm.
Identify pin 1 by the GND label at the end nearest the POWER OUT terminal.

Power, programming and status

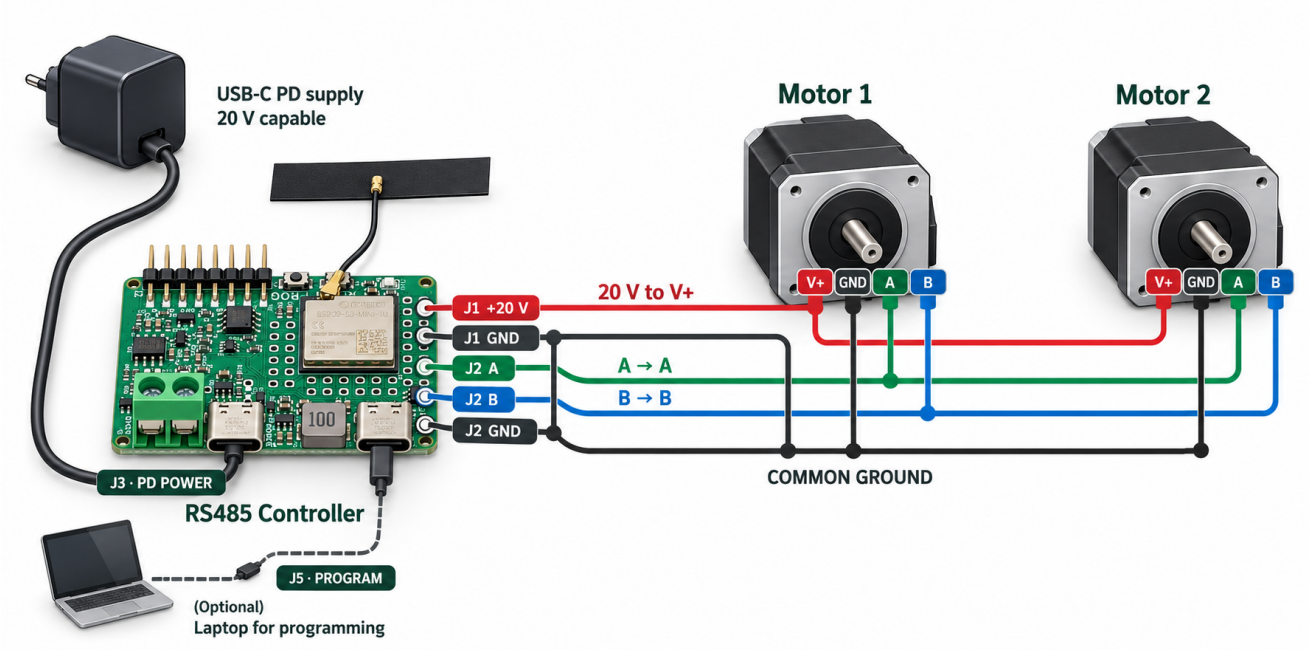
Reference	Function
J1 · POWER OUT	Two-way screw terminal, 5.0 mm pad spacing. Pin 1 is the positive PD rail; pin 2 is GND. Verify polarity on the board.
J3 · USB-C PD POWER IN	Power input and PD negotiation. USB data pins are unconnected; this socket does not program the ESP32-S3.
J5 · USB-C PROG	Native USB D–/D+ connect to GPIO19/GPIO20. Use a data-capable USB cable for programming.
S1 / S2	S1 RESET pulls EN low. S2 PROG pulls GPIO0 low for download mode.
D4 / D5 / D6	Yellow: logic power. Green: receive activity. Red: transmit activity. Activity indication depends on traffic.

Keep motor supply voltage off J2. The 3.3 V and 5 V pins are not motor-power terminals. Both bus groups and all GND points share the same electrical reference.

SYSTEM WIRING

Connect two servomotors

Example: one 20 V-capable USB-C PD source powers the controller and the motors through J1.



Functional illustration generated with AI and checked against the schematic. Terminal positions are schematic, not physical locations or motor connector pin order. Use the actual PCB labels and motor documentation.

Net	Controller connection	Motor 1 and Motor 2
+20 V · red	J1 pin 1	Each motor V+ / positive supply
GND · black	J1 pin 2 and J2 pin 1 or 4	Each motor GND / supply return
A · green	J2 pin 2 or 5	A to A, onward along the bus
B · blue	J2 pin 3 or 6	B to B, onward along the bus

1. With power off, connect both motor supply returns and controller GND to a common ground. Provide motor returns sized for motor current; the data-ground lead must not be the only motor-current return.

2. Connect A to A and B to B. Route A/B as a twisted pair along one bus, keeping branch leads short. Give each motor a distinct alias before normal addressed motion commands.

3. Check polarity and the J1 voltage, then power the system. Secure both motors before running discovery or movement examples. The supply, cable and distribution wiring must support the combined load.

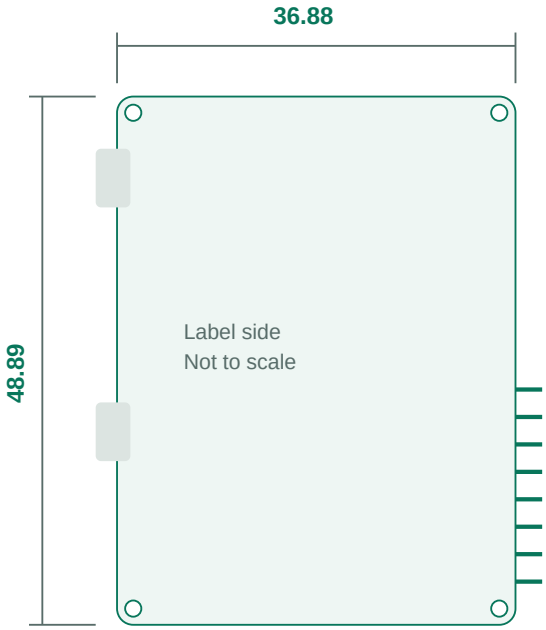
The M17 motor reference specifies 12–24 V motor operation. The controller standard PD request is 20 V. R5 already loads A/B with 220 Ω; choose any additional end termination for the complete bus rather than adding resistors at every motor. No fixed motor-count guarantee is specified.

MECHANICAL INTEGRATION

Dimensions & expansion

Nominal dimensions derived from the revision 0.5 KiCad board outline. Units: millimetres.

Item	Design value
Board outline	48.89 × 36.88 mm
PCB thickness	1.6 mm nominal
Corner cutouts	4 × Ø1.50 mm
Hole centre spacing	45.89 × 33.88 mm
Hole centre inset	1.50 mm from each adjacent edge
Unit mass	11.7 g including attached external antenna
Assembled envelope	Height, connector overhang and antenna/cable dimensions not specified



Outline excludes connectors and antenna. Hole dimensions are design values, not measured tolerances. The Ø1.5 mm corner cutouts are not M2/M3 clearance holes.

GPIO access

Connection	Notes
Expansion pads	Unpopulated GPIO pads expose ESP32-S3 signals. Use 3.3 V logic and verify each function against the schematic before connecting external circuits.
GPIO4 / GPIO5	Reserved by the onboard RS485 transmit / receive path in the supplied demos.
GPIO19 / GPIO20	Used by the native USB D– / D+ connection. Avoid external loads while using USB.
GPIO0 / 3 / 45 / 46	ESP32-S3 strapping pins. External circuitry can affect startup; GPIO0 is also connected to PROG.
External antenna	Attach the included antenna before using the radio. Allow clearance around the antenna and protect its small coaxial connector from cable strain.

Mount the PCB with clearance around underside components and connector solder joints. The open board has no specified ingress-protection rating. Use the KiCad geometry for enclosure design and confirm the assembled envelope on a physical unit.

BRING-UP & RESOURCES

Getting started

Program the ESP32-S3, establish communication, then enable motion deliberately.

Initial setup

1. **Check the wiring.** Connect the external antenna. With power off, check motor polarity and the A/B/GND connections on page 4.
2. **Connect the programming port.** Use J5 USB-C PROG with a data cable. J3 is the separate PD power socket.
3. **Install the software.** Use the Arduino ESP32 core and Gearotons Servomotor library. Select an ESP32-S3 target and configure native USB as appropriate for the board package.
4. **Enter download mode if needed.** Hold S2 PROG, press and release S1 RESET, then release PROG. Select the detected USB device and upload the application.
5. **Apply and verify motor power.** Use a suitable 20 V-capable PD source at J3. Verify J1 voltage first, then connect and power the motors. Match aliases and check discovery responses before enabling motion.

UART configuration

```
#include <Arduino.h>
void setup() {
  Serial1.begin(230400, SERIAL_8N1, 5, 4); // RX, TX
}
void loop() {}
```

This minimal snippet configures the RS485 UART and sends no motion commands. The repository contains separate motor-control, three-pass autodetection and ball-juggling examples. Build one program at a time; those examples can move motors.

Documentation & design files

Controller source: github.com/tomrodinger/RS485_Controller
Motor setup and protocol: tutorial.gearotons.com
Feedback: tutorial.gearotons.com/feedback

Specification basis. Revision 0.5 schematic, PCB and assembly BOM; supplied firmware examples; owner-provided mass and antenna statement; Gearotons M17 English datasheet v1.10. Manufacturer references: Espressif module datasheet, MaxLinear SP3485, TI TPS54202, and WCH CH221K configuration table in the repository CH224K.pdf.

Review status. DS v0.1, 26 October 2026. Pending owner review before publication. Values marked “not specified” require product-level confirmation; neither the drawings nor component data establish qualified board limits. Hardware/software source is supplied under the repository MIT licence.